

FORM 5 — AI Appendix — Case Study Attachment

i *Mandatory attachment for all case study submissions. To be completed by the candidate. Please be concise but specific. We evaluate your critical thinking, prompt strategy, and ability to refine AI outputs*

1. Tool Manifest

List all AI models and tools used. Please specify versions where applicable, e.g., Gemini Advanced, ChatGPT-4o, Midjourney v6.

Model / Tool	Primary Purpose of Use	Effectiveness (1-5) & Brief Why
Claude / Fable 5.1 (claude.ai chat)	Design phase: architecture design partner and failure mode enumeration.	5 — Argued the strongest case for the option I rejected and enumerated failure modes I hadn't considered; did not make decisions for me.
Claude Code (CLI) / Fable 5.1	Implementation, one session per module (scaffold + DB, domain, ingest, cache + load test), working under CLAUDE.md as a constraints file and ADR.md as the design. Also wrote the measurement scripts and the Postman collection.	4 — Implemented the design faithfully when constrained, and its plan-stage disagreements were its best contribution. Weaknesses: a subtle single-flight timing bug, style drift (nested closure factories), harness artefacts (attribution trailers, overwriting CLAUDE.md with ADR content).
Claude Code (CLI) / Opus 5	Implementation of a simple ui module which helps with manual testing and demonstration.	4 - built against the real API in one session. the UI server answered category names, the promotion list, product search, recent jobs and per-chunk status with its own SQL instead of asking for API endpoints

2. AI Tool Usage Approach

Detail your workflow. Do not paste full raw outputs; summarize your strategy and how you guided the AI.

Phase / Specific Task	Prompting Strategy & Context Provided	Human Refinement (How did you edit or iterate?)
Design before code (~2-3 hours, no code produced)	Provided the full case study up front, plus my background (CQRS read models, SQS pipelines at scale) so the AI could calibrate. Three deliberate techniques: (1) asked for a schema draft first so the Scenario B debate happened against concrete tables; (2) had the AI argue the strongest case for write-time materialization — the option I was rejecting — (3) asked for an ingestion pipeline sketch with realistic gaps left in, and found the holes myself before seeing the AI's list.	I made the Scenario B decision (read-time computation) and defended against the AI's best case for materialization; the exchange surfaced three arguments I hadn't articulated - the brief's 'instantly' requirement, scheduled-promotion handling and pricing logic spreading across write paths. I identified splitter crash recovery and DLQ replay ordering as Scenario A risks; the AI also enumerated more, I reviewed each fix and could re-derive it. Output: ADR draft and a CLAUDE.md
Implementation, one session per module (4 backend sessions + 1 UI session)	Fresh context per module; every session opens with "read CLAUDE.md and ADR.md", a fenced scope ("no ingest, no cache"), a plan-before-code requirement, and from Session 3 on an explicit invitation to disagree with the ADR in the plan. Small steps, diff + commit message per step, real test output at the end. State lives in the repo and the git log, not the conversation.	I approved or rejected every plan (rejected: Kysely schema-builder migrations; accepted: cursor scope binding, record-skip resume, dropping per-SKU cache deletes). Commit messages carry the "why". Reviewed the load-bearing files (migration, effective-price query, cursor, splitter, processor, cache) line by line; accepted boilerplate on the strength of the test suite. The UI was built with a different model (Opus 5) in a separate directory so it could not affect the backend build, tests, or startup.
Measurement-driven decisions	Asked for measurement scripts as deliverables, not claims: measure.sh runs the 500k ingest under a 128 MB heap cap with a forced-short timeout; the load script reports p95/p99 before/during/after a flash-sale version bump, with single-flight on and off. Pre-committed the decision rule in the prompt: "if cold-cache p99 with single-flight is acceptable, page warming	Two ADR deferred items closed by numbers (13.8 ms cold vs 11.0 ms warm) without a second debate. My proposed btree indexes were measured (EXPLAIN ANALYZE) and shown unnecessary — the EXCLUDE GiST indexes already served the lookup — so they were not added. A JIT anomaly (86 ms → 240 ms after autoanalyze) was diagnosed from EXPLAIN output and fixed at the pool.

	stays deferred with the number as justification."	
--	---	--

3. Judgement, Challenges & Verification

Highlight your critical thinking. How did you handle the limitations of the AI?

Challenge Encountered (e.g., hallucinations)	Judgement / Verification (How did you fact-check?)	Resolution (How did you fix the issue?)
AI asserted Postgres-specific behavior for the constraint and upsert design.	Checked each claim against the Postgres 16 docs and a throwaway psql session before adopting.	Confirmed; adopted
Migration runner created schema_migrations with IF NOT EXISTS before taking any lock	I asked "what happens with multiple instances?" — IF NOT EXISTS is not race-free; two replicas on an empty DB would collide on the catalog index and one would crash-loop.	One session-level advisory lock around the whole run; verified with 4 concurrent runners on a fresh DB: 1 applied, 3 skipped.
Integration tests ran against the development database	I noticed tests touched dev data, with row-level cleanup that later TRUNCATE-based tests would make unsafe.	TEST_DATABASE_URL, global setup that creates/migrates/truncates and refuses to run against dev; unit/integration split.
My own index proposal was wrong	I asked for two partial btree indexes on promotions; the agent ran EXPLAIN ANALYZE at 50k products and showed the EXCLUDE GIST indexes already BitmapOr'd, and the btree partials changed nothing (57 vs 59 ms).	Indexes not added; ADR #3 corrected. Lesson kept: measure before adding "the obvious index" to the hottest write table.

4. Overall Reflection

Estimated Ratio: *What percentage of the final case study would you estimate is directly AI-generated vs. human-crafted/heavily edited? (e.g., 30% AI / 70% Human)*

Code and tests: about 85% AI-generated, 15% mine. Every module was generated from the ADR in small reviewed steps; my share is the review of each diff, the corrections, and every decision about what to fix and how. ADR.md: roughly half and half. The decisions and the trade-off reasoning came out of a design session with the AI and I rewrote some of them by hand; the measurements and the production-experience paragraphs are mine, and I checked every failure-mode row against the code before it stayed in. CLAUDE.md: the constraints are mine; the text was drafted with the AI from the design session and I edited it. Overall, about 85% AI and 15% me, with my share concentrated where it matters: choosing the design, writing it (or having AI write it structurally) down before any code existed, and accepting or rejecting each change against it.

Key Takeaway: *Did using AI for this specific case study reveal any notable blind spots in the tools, or significantly alter your original approach? Briefly explain.*

The tool's own tests hid a real bug: the in-memory queue accepted any enqueue, so replay-failed passed its integration test while the production driver silently ignored the same call. The bug only surfaced when I asked for an end-to-end walk against the real stack. The lesson I now have in CLAUDE.md is that any behaviour depending on driver semantics needs a test against the real driver; a forgiving fake is worse than no test because it produces a green check. The first migration runner was transactional and idempotent and looked complete, and its test passed. I suspected a race condition and asked what happens when two API instances start at once, which is the normal case on a consumption plan, and the answer was a race: both runners read an empty schema_migrations, both apply 0001, one fails on CREATE TABLE. Even CREATE TABLE IF NOT EXISTS schema_migrations is not safe under concurrency. The fix was one session-level advisory lock held for the whole run, so the second instance blocks, then sees what the first recorded and skips it. The tool had optimised for the single-developer path it could see. Deployment shape is context the model does not have unless it is written down, which is why the ADR now states the constraints before any code is generated.
